

Git

Version control for beginners

Birgit Schröder

2026-09-27

Table of contents

Introduction	3
Ask the Git tutor	4
1 Basics	5
1 Why version control?	6
2 Setup	7
3 Your first repository	8
4 Working with a remote repository	11
5 .gitignore	13
6 Undoing changes	14
2 Teamwork	15
7 Branches	16
8 Merge conflicts	19
9 Pull requests and issues	20
3 Practice	21
10 Exercises	22
4 Background	23
11 Background: centralized vs. distributed version control	24
See also	25

Introduction

This book is a Git tutorial for beginners. It shows you how to save the history of a project, how to share it on GitHub and how to work in a team with branches and pull requests.

If you get stuck while reading, ask the Git tutor chat. The [next page](#) has the link.

- **Basics** (chapters 1 to 6): install Git, make commits, push and pull, and undo mistakes.
- **Teamwork** (chapters 7 to 9): branches, merge conflicts, pull requests and issues.
- **Practice** (chapter 10): exercises for pairs and a quiz to check what you learned.
- **Background** (chapter 11): why Git works the way it does. This chapter is optional.

The commands are shown for the terminal. PowerShell on Windows works fine. Where it helps, the book also shows how to do the same in VS Code.

Ask the Git tutor

Stuck with a command, an error message or an exercise? Ask the Git tutor chat. It knows this tutorial and answers in German or English. Keep it open while you work through the chapters:

💡 Git tutor

Hints and guiding questions first, in German or English.

▶ [Ask the Git tutor on app.novedu.at](https://app.novedu.at)



Scan the code, or type this address into your browser:

<https://app.novedu.at/vjy4114tum>

- Write what you already tried and what Git printed. Then you get a useful answer.
- For the exercises in **chapter 10**, the tutor asks you a question first instead of giving you the solution right away. If you are really stuck, say so and you get the answer.
- **Warning:** Never paste passwords or tokens into the chat.

Part 1

Basics

1 Why version control?

Imagine you work on a project with three team mates. Without a tool, you end up with files like `project-final.zip`, `project-final-v2.zip` and `project-final-REALLY-final.zip`. Nobody knows which version is the newest, and changes get lost.

Git solves this problem:

- Git saves a **history** of your project. You can always go back to an older version.
- Every change has an **author**, a **date** and a **message**. You can see who changed what and why.
- Many people can work on the same project **at the same time**.
- **GitHub** is a website that stores Git repositories online, so your team can share them.

Tip: Git and GitHub are not the same thing. **Git** is the program on your computer. **GitHub** is a website that hosts Git repositories.

1.1 Important words

Word	Meaning
Repository (repo)	A project folder that Git tracks. It contains all files and the complete history.
Commit	A saved snapshot of your project, with a message that describes the change.
Remote	A copy of the repository on a server, e.g. on GitHub. Its default name is origin .
Branch	A separate line of work. You can work on a branch without disturbing others.

2 Setup

You only need to do this **once** on each computer.

2.1 Install Git

- Windows: download and install Git from <https://git-scm.com>. Keep the default settings.
- Check that it works. Open a terminal (PowerShell or the terminal in VS Code) and type:

```
git --version
```

2.2 Tell Git who you are

Every commit stores your name and email address. Use the **same email address as in your GitHub account**, so GitHub can link your commits to you.

```
git config --global user.name "Max Mustermann"
git config --global user.email "max.mustermann@example.com"
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
```

- `init.defaultBranch main`: new repositories use `main` as the name of the first branch.
- `core.editor`: when Git needs a message from you (e.g. for a merge commit, see [chapter 7](#)), it opens VS Code. Write or check the message, save and close the tab. Without this setting, Git opens the editor **Vim** in the terminal, which is hard to use for beginners.

Tip: Stuck in Vim? Press `Esc`, type `:wq` and press `Enter`.

2.3 Sign in to GitHub

- Create an account on <https://github.com> if you don't have one.
- The first time you push to GitHub (see [chapter 4](#)), a browser window opens and asks you to sign in. After that, Git remembers you.

Tip: You cannot use your GitHub password directly in the terminal. Always sign in through the browser window.

2.4 VS Code

VS Code has Git built in. Open the **Source Control** view (the icon with the branches on the left side, or `Ctrl+Shift+G`). In this tutorial you learn the terminal commands, but you can do most steps in VS Code as well.

3 Your first repository

3.1 Get a repository

There are two ways to start:

a) **Copy an existing repository from GitHub** (this is what you do most of the time):

```
git clone <url>
cd <folder-name>
```

`git clone` downloads the complete repository **including its history** into a new folder.

b) **Turn an existing folder into a new repository:**

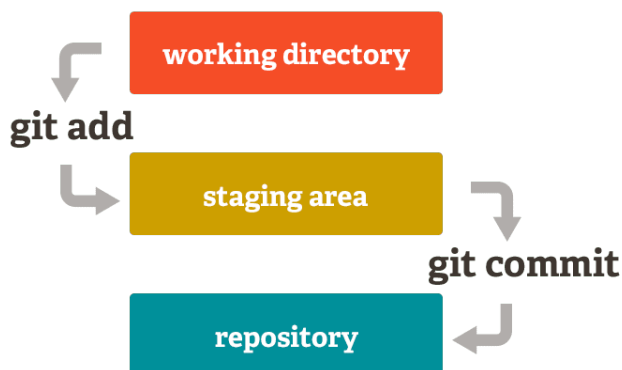
```
cd <your-project-folder>
git init
```

Both ways create a hidden folder called `.git`. This is where Git stores the history. You can see it with `ls -Force` (PowerShell) or `ls -a` (Git Bash).

Tip: If there is **no** `.git` folder in your project, Git commands don't work. Git will say `fatal: not a git repository`.

Warning: Never delete or change the `.git` folder by hand. If you delete it, the whole history is gone.

3.2 The three areas



Git has three areas for your files:

1. **Working directory:** the files you see and edit in your folder.
2. **Staging area:** the changes you have **chosen** for your next commit.
3. **Repository:** all commits that are saved in the history.

So saving a change in Git is a two-step process: first you **add** the change to the staging area, then you **commit** it.

Tip: A commit is saved **only on your computer**. Nobody else can see it until you **push** it (see [chapter 4](#)).

3.3 Check what has changed

Show which files are changed, new (untracked) or staged:

```
git status
```

Show the changed lines **before** you stage them:

```
git diff
```

Show the changed lines that are **already staged**:

```
git diff --staged
```

In VS Code, click on a file in the Source Control view to see the changes side by side.

3.4 Stage and commit

Add one file to the staging area:

```
git add <file>
```

Add all changed and new files to the staging area:

```
git add .
```

Save the staged changes as a new commit:

```
git commit -m "Add login form"
```

In VS Code: click + next to a file to stage it, write a message and click **Commit**.

3.5 Review before you commit

Tip: We always review our changes during staging!

Don't just type `git add .` without looking. Instead:

1. Look at each changed file with `git diff` or in VS Code.
2. Is the change correct and complete? Then stage it.
3. Stage all files that belong to **one topic** and commit them together.
4. Repeat with the next topic.

Example: you changed 27 files. 20 of them are for the new login page and 7 fix a bug in the menu. Make **two commits**, one for each topic.

3.6 Write good commit messages

A good commit message tells your team **what** changed. Your team mates should understand it without looking at the code.

✗ Bad	✓ Good
update	Add validation to registration form
fix	Fix crash when user list is empty
asdf	Remove unused images from pics folder
changes from Monday	Translate start page to English

Tips:

- Start with a verb: **Add, Fix, Remove, Change, Update**.
- Keep the first line short (about 50 characters).
- If the commit belongs to a GitHub issue, add the issue number (see [chapter 9](#)).

3.7 Look at the history

```
git log --oneline
```

Each line is one commit: a short **commit ID** (e.g. 8756b24) and the message. Press `q` to leave the list.

To see the history of all branches as a graph:

```
git log --oneline --graph --all
```

3.8 Look at a commit

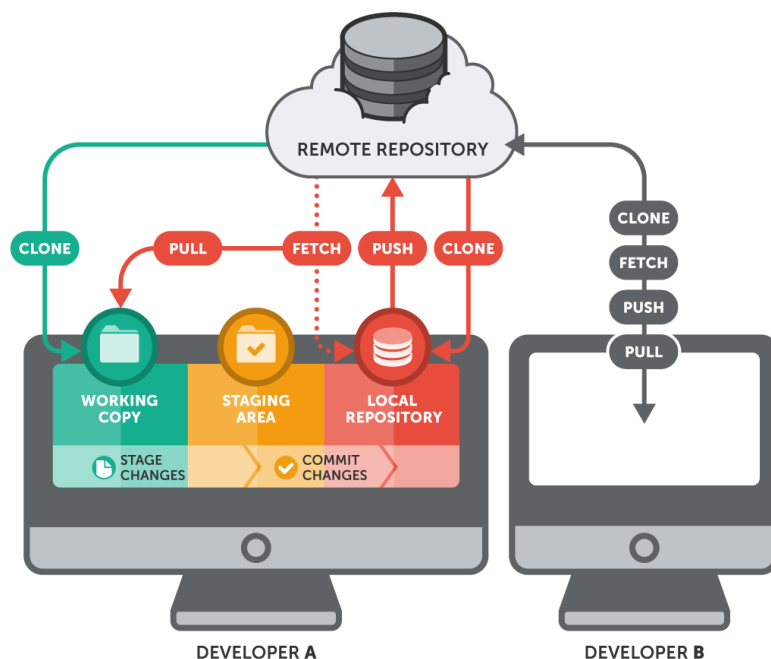
Every commit has an ID, e.g. 8756b24. Git also has short names for commits:

- **HEAD** is the commit you are on right now (usually your newest commit).
- **HEAD~1** is one commit before **HEAD**, **HEAD~2** is two commits before, and so on.

Show which lines a commit changed:

```
git show HEAD  
git show 8756b24
```

4 Working with a remote repository



Reference: <https://medium.com/mindorks/what-is-git-commit-push-pull-log-aliases-fetch-config-clone-56bc52a3601c>

Your commits are only on your computer. To share them, you use a **remote** repository, e.g. on GitHub.

4.1 Push: send your commits to GitHub

```
git push
```

When you push a new branch for the first time, you have to tell Git where to send it:

```
git push -u origin <branch-name>
```

After that, `git push` is enough. In VS Code, click **Publish Branch** or **Sync Changes**.

4.2 Pull: get the commits of your team mates

```
git pull
```

Tip: Pull before you start working. Then you always work with the newest version.

4.3 What happens if your push is rejected?

If a team mate pushed something since your last pull, GitHub rejects your push:

```
! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/...'
hint: Updates were rejected because the remote contains work that you do not have locally.
```

This is **not** an error in your code. Git just wants you to get the new commits first:

1. `git pull` (Git combines their commits with yours).
2. `git push`

Usually this works without problems. If you and your team mate changed **the same lines** of the same file, you get a **merge conflict** (see [chapter 8](#)).

4.4 Daily workflow

<code>git pull</code>	# 1. get the newest version
<code># ... edit your files ...</code>	
<code>git status</code>	# 2. what has changed?
<code>git diff</code>	# 3. review your changes
<code>git add <file></code>	# 4. stage
<code>git commit -m "Message"</code>	# 5. commit
<code>git push</code>	# 6. share

5 .gitignore

Some files should **never** go into the repository, for example:

- installed libraries (`node_modules/`)
- build output (`dist/`, `bin/`, `obj/`)
- IDE settings and temporary files (`.idea/`, `*.log`)
- files with passwords or secrets (`.env`)

Create a file called `.gitignore` in the main folder of the repository. Write one file name or pattern in each line:

```
node_modules/  
dist/  
*.log  
.env
```

Git ignores these files. They don't show up in `git status` anymore.

Tip: Use <https://www.toptal.com/developers/gitignore> to create a `.gitignore` file for your programming language and IDE.

Warning: `.gitignore` only works for files that are **not yet committed**. Create it at the beginning of the project.

5.1 You already committed a file by mistake?

First add the file to `.gitignore`. Then remove it from the repository, but keep it on your computer:

```
git rm --cached <file>           # for a file  
git rm -r --cached node_modules/ # for a folder  
git commit -m "Remove node_modules from repository"
```

`--cached` means: remove it from Git only, not from your disk.

Warning: The file is still in the **old commits**. If you committed a password or another secret, removing the file is not enough. **Change the password!**

Git also ignores **empty folders**. A folder only appears in the repository when there is at least one file in it.

6 Undoing changes

Everybody makes mistakes. Here is how to fix them:

Table 6.1: warning: `git restore <file>` and `git clean` delete your changes for good. There is no undo. Run `git clean -nd` first: it only **shows** which files and folders would be deleted.

Situation	Command
I changed a file and want the last committed version back.	<code>git restore <file></code>
I created new files and want to delete them again.	<code>git clean -fd</code>
I want a file back as it was 2 commits ago.	<code>git restore --source=HEAD~2 <file></code>
I staged a file by mistake.	<code>git restore --staged <file></code>
My last commit message is wrong, and I have not pushed yet.	<code>git commit --amend -m "New message"</code>
I forgot a file in my last commit, and I have not pushed yet.	<code>git add <file></code> , then <code>git commit --amend --no-edit</code>
A commit is wrong and I already pushed it.	<code>git revert <commit-id></code>

Tip: `git restore .` only resets files that Git already knows. New (untracked) files stay. Use `git clean` for them.

Tip: After `git restore --source=...`, the old version is in your working directory like a normal change. Commit it if you want to keep it.

Warning: Only use `--amend` for commits that you have **not pushed** yet. For pushed commits, use `git revert`. It creates a **new** commit that undoes the old one, so the history of your team mates stays correct.

6.1 Look at an old version: detached HEAD

You can go back in time and look at the whole project as it was in an old commit:

```
git switch --detach <commit-id>
```

Many tutorials use `git checkout <commit-id>` for this. It does the same.

Now you are in the **detached HEAD** state. This means you are **not on a branch**. `git status` shows `HEAD detached at <commit-id>`, and VS Code shows the commit ID instead of a branch name in the bottom left corner.

You can look at the files and run the code, but **don't work here**. Commits in this state don't belong to any branch, and they get lost when you switch away.

Go back to your branch:

```
git switch main
```

Tip: Did you make commits in detached HEAD and want to keep them? Create a branch **before** you switch away: `git switch -c <new-branch-name>`.

6.2 Put changes aside: stash

Sometimes you have unfinished changes, but you need to pull or switch to another branch. Git may not let you do that. Put your changes aside with:

```
git stash
```

Now your working directory is clean. Do what you need to do (e.g. `git pull`). Then get your changes back:

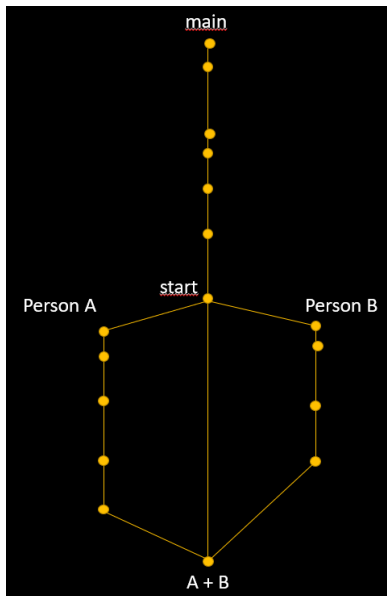
```
git stash pop
```

Tip: If your stashed changes and the pulled changes touch the same lines, you get a merge conflict during `git stash pop`. Solve it like any other conflict (see [chapter 8](#)).

Part 2

Teamwork

7 Branches



If everybody works directly on **main**, you constantly get in each other's way. That's why each team member works on their own **branch**:

- **main** is the main line. It should always contain a **working** version of the project.
- For each new feature or bug fix, you create a new branch from **main**.
- You commit on your branch. Your team mates don't see your unfinished work on **main**.
- When the work is done, the branch is **merged** back into **main**.

In the picture, person A and person B both start from the same version of **main**. Each of them works on their own branch. In the end, both branches are merged, and **main** contains the work of A **and** B.

Tip: A branch doesn't prevent merge conflicts. It only moves them to the moment when you merge. The longer a branch lives, the more conflicts you get. So **merge often**.

7.1 Create a branch

First make sure you are on **main** and have the newest version:

```
git switch main
git pull
```

Create a new branch and switch to it:

```
git switch -c <branch-name>
```

Choose a name that describes the work, e.g. **login-page** or **12-fix-menu-crash**.

Now edit, stage and commit as usual. Share the branch on GitHub:

```
git push -u origin <branch-name>
```

7.2 Switch between branches

Show all local branches. The ***** marks the branch you are on:

```
git branch
```

Also show the branches on GitHub (they start with **remotes/origin/**):

```
git branch -a
```

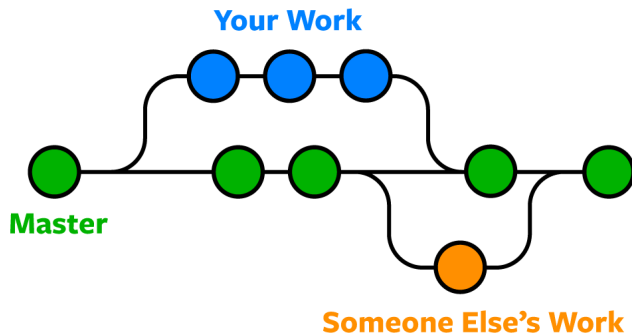
Switch to another branch:

```
git switch <branch-name>
```

In VS Code, the name of the current branch is shown in the bottom left corner. Click on it to switch branches.

Tip: Commit or stash your changes before you switch branches.

7.3 Keep your branch up to date



Reference: <https://www.nobledesktop.com/learn/git/git-branches> (in this picture, `main` has its old name `master`)

While you work on your branch, your team mates merge their work into `main`. Get their changes into your branch regularly:

```
git switch main
git pull          # get the newest main from GitHub
git switch <branch-name>
git merge main    # copy the changes from main into your branch
```

If there are conflicts, solve them now (see [chapter 8](#)). Then push your branch:

```
git push
```

It's better to solve conflicts on **your** branch than on `main`.

7.4 Merge your branch into main

In a team, we merge branches with a **pull request** on GitHub (see [chapter 9](#)). Then your team can review your work first.

Without a pull request, you can merge on your computer:

```
git switch main
git pull
git merge <branch-name>
git push
```

Tip: After every merge, test `main`. Does everything still work?

7.5 Two kinds of merges

When you merge, Git sometimes prints **Fast-forward** and sometimes creates a new commit. Both are fine.

Fast-forward: `main` has **no new commits** since you created your branch. Git simply moves `main` forward to the newest commit of your branch. No new commit is created.

Merge commit: `main` and your branch both have new commits. Git combines them in a new **merge commit**. In `git log --oneline --graph` you see two lines that join. Git opens your editor for the commit message: check it, save and close it.

Tip: On GitHub, the **Merge pull request** button creates a merge commit by default. So the history always shows which pull request brought in which changes.

7.6 What about rebase?

Maybe you have seen `git rebase` in a tutorial or on GitHub. It is another way to get the changes from `main` into your branch:

```
git switch <branch-name>
git rebase main
```

Rebase takes your commits and puts them **on top of** the newest **main**. The history becomes one straight line, without a merge commit.

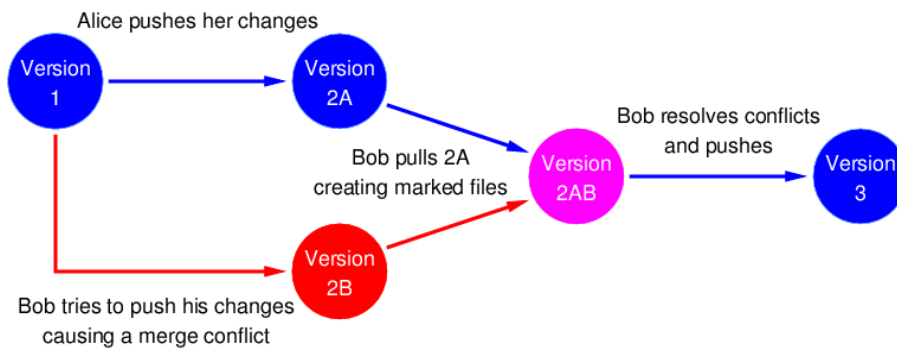
In this course we use `git merge` (see [Keep your branch up to date](#)). Merging shows the history as it really happened, and it never changes commits that already exist.

Warning: Never rebase commits that you have already pushed, or that somebody else has. Rebase throws your old commits away and creates **new** ones. To push them you would need `git push --force`, and that deletes the work of your team mates.

Tip: Started a rebase and something went wrong? `git rebase --abort` brings back the state from before.

Tip: On GitHub, the **Merge** button has a dropdown with three options: *Create a merge commit* (what we use), *Squash and merge* (all commits of the branch become one) and *Rebase and merge* (puts the commits on top of **main**). Use the first one, unless your team agreed on something else.

8 Merge conflicts



Reference: <http://www.cs.utsa.edu/~cs3443/git/merge-conflicts.html>

A **merge conflict** happens when two people change **the same lines** of the same file. Git can't decide which version is correct, so **you** have to decide.

Conflicts can happen during `git pull`, `git merge` or `git stash pop`. Git tells you which files have conflicts:

```
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

8.1 What a conflict looks like

Git writes both versions into the file and marks them:

```
<<<<<<< HEAD
<h1>Welcome to our shop</h1>
=====
<h1>Welcome to the HTL shop</h1>
>>>>>>> main
```

- Between <<<<<<< and =====: **your** version (VS Code calls it **Current Change**).
- Between ===== and >>>>>>>: the version **coming in** (VS Code calls it **Incoming Change**).

8.2 Solve a conflict

1. Open the file. VS Code shows buttons above each conflict: **Accept Current Change**, **Accept Incoming Change**, **Accept Both Changes**.
2. Choose a button, or edit the lines by hand until the code is correct.
3. Make sure **all** markers (<<<<<<<, =====, >>>>>>>) are gone.
4. Test your code. Does it still work?
5. Stage and commit the solved files:

```
git add <file>
git commit -m "Merge main into login-page"
```

Tip: Talk to your team mate if you are not sure which version is correct.

Tip: Want to start over? `git merge --abort` cancels the merge and brings back the state from before.

9 Pull requests and issues

9.1 Pull requests

A **pull request** (PR) is a request on GitHub: “Please merge my branch into **main**.”

Before the merge, your team can:

- look at all your changes,
- write comments and ask questions,
- ask you to improve something. You simply commit and push to the same branch, and the pull request updates automatically.

How to create a pull request:

1. Update your branch with **main** and push it (see **Keep your branch up to date**).
2. Open the repository on GitHub. Click **Pull requests** → **New pull request**.
3. Choose **main** as **base** and your branch as **compare**.
4. Write a title and a short description of what you did. Click **Create pull request**.
5. A team mate reviews the changes and approves them.
6. Click **Merge pull request**. Then click **Delete branch** to remove the branch on GitHub.
7. On your computer, switch back to **main**, get the merge and delete your local branch:

```
git switch main
git pull
git branch -d <branch-name>
```

9.2 Link commits and pull requests to issues

In our projects, we plan the work with **GitHub issues**. Each issue has a number, e.g. **#12**.

Mention the issue in a commit message. The commit then appears in the history of the issue:

```
git commit -m "Add input validation to login form #12"
```

Close the issue automatically with a keyword like **closes**, **fixes** or **resolves**. You can use it in a commit message or in the description of the pull request:

```
git commit -m "Fix crash when user list is empty, fixes #12"
```

The issue is closed as soon as the commit or pull request is **merged into main**.

Tip: You can also create a branch directly from an issue: open the issue on GitHub and click **Create a branch** under **Development**. GitHub names the branch after the issue, e.g. **12-fix-menu-crash**.

Part 3

Practice

10 Exercises

Work in pairs (person A and person B). If you are stuck, ask the [Git tutor](#).

1. **Setup:** both of you set up Git ([chapter 2](#)). Check your settings with `git config --global --list`.
2. **First commits:** person A creates a new repository on GitHub and adds person B as a collaborator. Both of you clone it. Each of you creates a file with your name, commits it and pushes it. Look at the history with `git log --oneline`.
3. **Rejected push:** both of you change a **different** file without pulling first. Person A pushes, then person B pushes. What happens? Fix it with `git pull` and `git push`.
4. **Merge conflict:** both of you change the **same line** in the same file. Person A pushes first. Person B pulls and solves the conflict in VS Code.
5. **Branch and pull request:** person A creates an issue “Add a README”. Person B creates a branch for it, writes the README, commits with `closees #<number>` and opens a pull request. Person A reviews and merges it. Is the issue closed now?
6. **Undo:** change a file and throw the change away with `git restore`. Make a commit, push it and undo it with `git revert`.
7. **Committed by mistake:** create a folder `node_modules` with a file in it, commit and push it. Remove it from the repository with `.gitignore` and `git rm -r --cached`. Check that the folder is still on your disk, but gone on GitHub.
8. **Time travel:** look at the project as it was two commits ago with `git switch --detach HEAD~2`. What does `git status` say? Go back with `git switch main`. Then get one file back from two commits ago with `git restore --source=HEAD~2 <file>` and commit it.

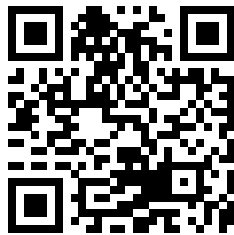
Practice more on <https://learngitbranching.js.org>. It shows what happens to your branches as a picture.

10.1 Check what you learned

Answer eight questions about this tutorial in the Novedu quiz. You write your answers in your own words, in English or German, and you get feedback for each answer:

 Quiz: Git basics

► [Take the quiz on app.novedu.at](https://app.novedu.at)



Scan the code, or type this address into your browser:

<https://app.novedu.at/xmen1hvm3x>

Tip: The questions are about the **why**, not only about the commands. Explain your answer like you would explain it to a team mate.

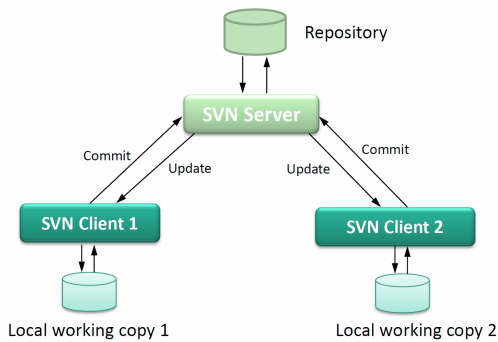
Part 4

Background

11 Background: centralized vs. distributed version control

This chapter is optional. It explains why Git works the way it does.

11.1 Centralized version control (e.g. Subversion)

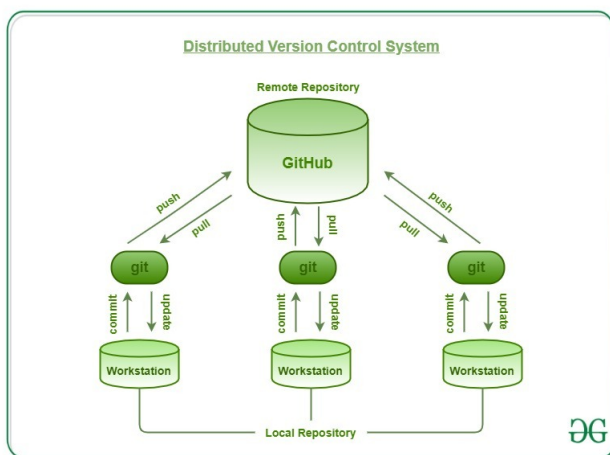


Reference: <https://codentrick.com/architecture-and-design-of-subversion-svn-part-1/>

Older tools like **Subversion (SVN)** use a **single central server**:

- The history is stored **only on the server**.
- Each developer only has a **working copy** of the files (`svn checkout`), without the history.
- A **commit** sends the changes **directly to the server**. Without a network connection you can't commit.

11.2 Distributed version control (Git)



Reference: <https://www.geeksforgeeks.org/git-features/>

With Git, **every developer has a complete copy** of the repository, including the full history:

- You can commit, look at the history and create branches **without a network connection**.
- You decide **when** to share your commits with `git push`.
- If the server breaks, every clone is a complete backup.
- You can push the same repository to **several** remotes, e.g. GitHub and a school server.

See also

- Pro Git book (free, also available in German): <https://git-scm.com/book>
- Learn Git Branching (interactive): <https://learngitbranching.js.org>
- Video tutorial: <https://www.youtube.com/watch?v=M2tz-GASzR8>